

Spring Security Integration

By Abderrahmen Masmoudi

application.properties.xml

spring.application.name=sportdec

spring.datasource.url=jdbc:mysql://localhost:3306/nameOfDB?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC

spring.datasource.username=root

spring.datasource.password=

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

Show or not log for each sql query

spring.jpa.show-sql=true

Hibernate ddl auto (create, create-drop, update)

spring.jpa.hibernate.ddl-auto=update

The SQL dialect makes Hibernate generate better SQL for the chosen database

spring.jpa.hibernate.dialect=org.hibernate.dialect.MySQLDialect

Naming strategy

spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl

server.port=9000

CORS Configuration

Créer un nouveau package nommé : config

Ajouter la classe suivante :

```
@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                    .allowedOrigins("")
                    .allowedMethods("GET", "POST", "PUT", "DELETE")
                    .allowedHeaders("");
            }
        };
    }
}
```

Security Configuration

Ajouter la classe suivante :

```
@Configuration
```

```
@EnableWebSecurity
```

```
public class SecurityConfig {
```

```
    // Code à voir
```

```
}
```

Relation entre Player et Team

— — —

Player Model :

@ManyToOne

@JoinColumn(name = "team_id")

@JsonIgnoreProperties("players")

private Team team;

Team Model :

@OneToMany(mappedBy = "team", cascade = CascadeType.ALL)

@JsonIgnoreProperties("team")

private List<Player> players;

Ajouter les dépendances nécessaires (1)

— — —

```
<!-- Dépendances Spring Security -->
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-security</artifactId>
```

```
    </dependency>
```

```
    <dependency>
```

```
        <groupId>org.springframework.security</groupId>
```

```
        <artifactId>spring-security-crypto</artifactId>
```

```
    </dependency>
```

```
    <dependency>
```

```
        <groupId>org.springframework.security</groupId>
```

```
        <artifactId>spring-security-test</artifactId>
```

```
        <scope>test</scope> <!-- Scope de test -->
```

```
    </dependency>
```

Ajouter les dépendances nécessaires (2)

— — —

```
<!-- Dépendances JWT pour la gestion des tokens -->
```

```
  <dependency>
```

```
    <groupId>io.jsonwebtoken</groupId>
```

```
    <artifactId>jjwt-api</artifactId>
```

```
    <version>0.11.5</version>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>io.jsonwebtoken</groupId>
```

```
    <artifactId>jjwt-impl</artifactId>
```

```
    <version>0.11.5</version>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>io.jsonwebtoken</groupId>
```

```
    <artifactId>jjwt-jackson</artifactId>
```

```
    <version>0.11.5</version>
```

```
  </dependency>
```

Ajouter les dépendances nécessaires (3)

— — —

```
<!-- Starter pour l'envoi d'emails -->  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-mail</artifactId>  
    <version>3.2.5</version>  
  </dependency>
```

NB :

Jackson est une bibliothèque puissante et très flexible pour travailler avec JSON. Elle permet une personnalisation lors de la conversion des objets Java en JSON et inversement.

Création des modèles nécessaires(1)

— — —

1. User Model

Assurer la relation @ManyToMany

```
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "user_roles",
    joinColumns = @JoinColumn(name = "user_id"),
    inverseJoinColumns = @JoinColumn(name = "role_id")
)
private List<Role> roles = new ArrayList<>();
```

Création des modèles nécessaires(2)

1. Role Model

Assurer la relation @ManyToMany

```
@Column(unique = true, nullable = false)

private String name;


@ManyToMany(mappedBy = "roles")

@JsonIgnore

private List<User> users = new ArrayList<>();
```

NB : Ajouter les rôles dans la DB (Admin, Client, Manager ...)

Création des repositories nécessaires(1)

— — —

1. User Repo

@Repository

```
public interface UserRepository extends JpaRepository<User, Long> {
```

```
    @Query("SELECT u FROM User u WHERE u.email = :email")
```

```
    User findUserByEmail(@Param("email") String email);
```

```
}
```

Création des repositories nécessaires(2)

— — —

1. Role Repo

@Repository

```
public interface RoleRepository extends JpaRepository<Role, Long> {  
    Optional<Role> findByName(String name);  
}
```

Création du service

1. User Service

```
User addUser(User user);  
  
User findUserById(Long id);  
  
void deleteUser(Long id);  
  
User getUserByEmail(String email);  
  
boolean changePassword(Long id, ChangePassword changePassword);
```

Création du service Implementation (1)

— — —

1. UserServiceImpl

```
@Autowired
```

```
UserRepository userRepository;
```

```
@Autowired
```

```
private PasswordEncoder passwordEncoder;
```

Création du service Implementation (2)

— — —

1. UserServiceImpl

```
@Override  
public User addUser(User user) {  
    // crypt password  
    String encodedPassword = passwordEncoder.encode(user.getPwd());  
    user.setPwd(encodedPassword);  
    return userRepository.save(user);  
}
```

Création du service Implementation (3)

1. UserServiceImpl

```
@Override

public User findUserById(Long id) {

    return userRepository.findById(id).get();

}

@Override

public void deleteUser(Long id) {

    userRepository.deleteById(id);

}

public User getUserByEmail(String email) {

    return userRepository.findUserByEmail(email);

}
```

Création du service Implementation (4)

— — —

```
public boolean changePassword(Long id, ChangePassword changePasword) {  
  
    // Retrieve the user from the database  
  
    Optional<User> optionalEntreprise = userRepository.findById(id);  
  
    if (optionalEntreprise.isPresent()) {  
  
        User user = optionalEntreprise.get();  
  
        // Check if the old password matches the current password  
  
        if (passwordEncoder.matches(changePasword.getOldPassword(), user.getPwd())) {  
  
            // Encode the new password  
  
            String encodedNewPassword = passwordEncoder.encode(changePasword.getNewPassword());  
  
            user.setPwd(encodedNewPassword);  
  
            // Save the updated user  
  
            userRepository.save(user);  
  
            return true; // Password change was successful  
  
        } else {  
  
            return false; // Old password does not match  
  
        }  
  
    } else {  
  
        throw new EntityNotFoundException("User not found with ID: " + id);  
  
    }  
  
}
```

Création du service Implementation (5)

— — — 2. UserDetailsServiceImpl

@Service

```
public class UserDetailsServiceImpl implements UserDetailsService {  
    @Autowired  
    UserRepository userRepository;  
    @Override  
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {  
        User user=userRepository.findUserByEmail(email);  
        if (user == null) {  
            throw new UsernameNotFoundException("User not found with email: " + email);  
        }  
        return org.springframework.security.core.userdetails.User  
            .withUsername(email)  
            .password(user.getPwd())  
            .build();  
    }  
}
```

Création du package DTO (class 1)

— — — Sous le package principal, créer un package DTO

```
1. public class ChangePassword {
2.     private String oldPassword;
3.     private String newPassword;
4.     /**
5.      * @return the oldPassword
6.      */
7.     public String getOldPassword() {
8.         return oldPassword;
9.     }
10.    /**
11.     * @param oldPassword the oldPassword to set
12.     */
13.    public void setOldPassword(String oldPassword) {
14.        this.oldPassword = oldPassword;
15.    }
16.    /**
17.     * @return the newPassword
18.     */
19.    public String getNewPassword() {
20.        return newPassword;
21.    }
22.    /**
23.     * @param newPassword the newPassword to set
24.     */
25.    public void setNewPassword(String newPassword) {
26.        this.newPassword = newPassword;
27.    }
28.
29.
30. }
31.
32.
```

Création du package DTO (class 2)

— — —

Sous le package principal, créer un package DTO

```
1. public class LoginRequest{
2.     private String email;
3.     private String pwd;
4.
5.     // Getters and setters
6.
7.
8. }
```

Création du package DTO (class 3)

— — —

Sous le package principal, créer un package DTO

```
1.  public class SignUpRequest {
2.      private String firstName;
3.      private String lastNme;
4.      private String email;
5.      private String pwd;
6.      private Long phone;
7.      private List<String> roles;
8.
9.
10.     // Gettres and setters
11.
12.
13. }
```

Créer UserDetailsServiceImpl

— — —

@Service

```
public class UserDetailsServiceImpl implements UserDetailsService {
```

```
    @Autowired
```

```
    UserRepository userRepository;
```

```
    @Override
```

```
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
```

```
        User user = userRepository.findUserByEmail(email);
```

```
        if (user == null) {
```

```
            throw new UsernameNotFoundException("User not found with email: " + email);
```

```
        }
```

```
        return
```

```
        org.springframework.security.core.userdetails.User.withUsername(email).password(user.getPwd())  
            .build();
```

```
    }
```

Integration de JWT

Ajouter un package nommé **auth** sous le package principal

1. Ajouter la classe : JwtFilter
2. Ajouter la classe : JwtUtil

Lien: <https://mega.nz/folder/BDNDSa7Q#INAmXCvie0y90i95C1GFiw>

Integration de Security Configuration

Sous le package Config

1. Ajouter la classe : SecurityConfig

Lien

<https://mega.nz/folder/BDNDSa7Q#INAmXCvie0y90i95C1GFiw>

Ajouter cette dépendance (pom.xml)

— — —

```
<dependency>
```

```
    <groupId>org.apache.commons</groupId>
```

```
    <artifactId>commons-lang3</artifactId>
```

```
    <version>3.12.0</version>
```

```
</dependency>
```

Ajouter UserController

Ajouter la classe UserController sous le package
Controllers:

Lien

<https://mega.nz/folder/BDNDSa7Q#INAmXCvie0y90i95C1GFiw>

More Info about the concept (1)

— — —

```
Map<String, Object> map = new HashMap<>();
```

Map<String, Object> : Il s'agit d'une interface en Java qui représente une collection d'entrées clé-valeur.

new HashMap<>() : C'est une implémentation concrète de l'interface **Map**. Un **HashMap** est une structure de données qui stocke des paires clé-valeur et permet une recherche rapide des valeurs par leurs clés.

@Component

@Component est une annotation générale, mais il existe d'autres annotations spécifiques qui dérivent de **@Component**, telles que :

- **@Service** : utilisée pour marquer les classes de service.
- **@Repository** : utilisée pour marquer les classes de repository (accès aux données).
- **@Controller** : utilisée pour marquer les classes qui gèrent les requêtes HTTP dans une application web.

Différence entre @Bean et @Component

@Component :

- C'est une annotation de stéréotype qui indique à Spring qu'une classe est un composant à gérer.

@Bean

- C'est une annotation utilisée pour déclarer explicitement un bean dans une méthode d'une classe de configuration.

@Component :

- Utilisé pour **annoter une classe directement**.
- Le bean est automatiquement détecté par Spring lors du scan des composants (**@ComponentScan**).

@Bean :

- Utilisé pour **annoter une méthode** dans une classe de configuration (**@Configuration**).
- Permet de définir manuellement des beans, notamment pour des objets complexes ou issus de bibliothèques tierces.

Authentication

— — —

Dans Spring Security, l'interface `Authentication` est l'un des concepts clés pour gérer l'authentification des utilisateurs. Elle représente le **principal** (souvent l'utilisateur) qui est actuellement authentifié, ainsi que les **détails de l'authentification**. Cette interface fait partie du framework de sécurité et est utilisée pour transmettre et manipuler les informations d'authentification tout au long du cycle de vie de la requête.

UsernamePasswordAuthenticationToken

— — —

UsernamePasswordAuthenticationToken est une classe fournie par Spring Security qui implémente l'interface Authentication. Elle est principalement utilisée pour encapsuler les détails d'authentification d'un utilisateur lorsqu'il s'authentifie via un nom d'utilisateur(email, tel, cin ...) et un mot de passe.

UserDetailsService

— — —

L'interface `UserDetailsService` fait partie de Spring Security et est utilisée pour récupérer les informations d'authentification d'un utilisateur dans le cadre du processus d'authentification. C'est une interface essentielle qui permet à Spring Security de **charger les informations d'un utilisateur** (comme son nom d'utilisateur, mot de passe et rôles) à partir d'une source de données (comme une base de données) en vue d'authentifier l'utilisateur.

TOKEN

— — —

Le token JWT est encodé en Base64URL, ce qui signifie que n'importe qui peut le décoder et voir le contenu du payload sans avoir besoin de la clé secrète.

Cependant, la partie cruciale qui ne doit pas être exposée est la signature, car elle garantit que le token n'a pas été altéré. Si quelqu'un modifie le payload, la signature ne correspondra plus et le serveur rejettera le token lors de la validation.

Ce qu'il faut retenir : Le payload n'est pas chiffré, il est juste encodé, donc visible par n'importe qui. Si vous avez des informations sensibles, vous ne devriez pas les mettre dans le payload d'un JWT.

User Service (FE)

```
import { HttpClient } from "@angular/common/http";
import { Injectable } from "@angular/core";

@Injectable({
  providedIn: "root",
})
export class UserService {
  userURL: string = "http://localhost:9000/api/users"
  constructor(private http: HttpClient) { }

  // Add User
  // user = {firstName, lastName, email, pwd, phone, roles}
  signup(user: any) {
    return this.http.post(this.userURL + "/signup", user);
  }

  // connexion
  // user = {email, pwd}
  login(user: any) {
    return this.http.post(this.userURL + "/login", user);
  }
}
```

Interceptor Service

— — —

```
import { HttpHandler, HttpInterceptor, HttpRequest } from
'@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class InterceptorService implements HttpInterceptor {
  constructor() { }
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    let authReq = req;
    // Get token from LS

    let token = localStorage.getItem('jwt');
    console.log("here token", token);

    if (token) {
      authReq = req.clone({
        headers: req.headers.set("Authorization", "Bearer " +
token),
      });
      console.log("Here request", authReq);

    }
    return next.handle(authReq);
  }
}
```

Login Class

— — —

```
export class LoginComponent implements OnInit {
  user: any = {};
  errorMsg!: string;

  constructor(private userService: UserService, private router: Router)
  { }

  ngOnInit() {}
  login() {
    console.log("here object", this.user);
    this.userService.login(this.user).subscribe(
      (response: any) => {
        console.log("Here response after login", response);
        if (response.token) {
          localStorage.setItem('jwt', response.token);
          const decoded: any = jwtDecode(response.token);
          console.log("here decoded", decoded);
          if (decoded.roles.includes("admin")) {
            this.router.navigate(['admin']);
          } else {
            this.router.navigate(['/']);
          }
        }
      },
      (error: any) => {
        console.log("here error", error);
        this.errorMsg = "Please check Email/Pwd";
      }
    )
  }
}
```

Signup Method

— — —

```
signup() {  
    console.log("Here user object", this.signupForm.value);  
    this.signupForm.value.roles = [ "manager"];  
    this.userService.signup(this.signupForm.value).subscribe(  
        (response) => {  
            console.log("Here response after signup", response);  
        }  
    );  
}
```

App Module

```
providers: [  
  { provide: HTTP_INTERCEPTORS, useClass: InterceptorService, multi: true }  
],
```

Modifier la classe SecurityConfig (dans le package Config)

— — —

Changer la méthode dans la classe SecurityConfig.java par :

```
@Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.cors().and().csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests((requests) ->
requests.requestMatchers("/api/users/**").permitAll().anyRequest()

                .authenticated())

            .sessionManagement(manager ->
manager.sessionCreationPolicy(STATELESS))

            .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);
    }
```